

The Value of Personalization in Large Language Model Responses

Andrey Fradkin¹, Yan (Stella) Si², Giorgos Zervas^{1,2}

¹Questrom School of Business, Boston University

²Faculty of Computing & Data Sciences, Boston University

September 2026

Abstract

We study how personalized LLM routing increases end-user utility. To do so, we estimate a random-coefficients demand model on Arena preference data, in which utility depends on the judged accuracy of a response as well as its style, the prompt, model identity, and a user’s latent tastes, and simulate the effects of a router that picks a model based on the user, prompt, or user-prompt pair. Accuracy is the largest single driver of preference, with diminishing returns, and style preferences are not displaced by it. A prompt only router raises predicted mean per-user utility over the best single model, attaining a 53.8% model-implied head-to-head win rate. Allowing the router to personalize to each user’s own tastes, rather than serving everyone the average user’s pick, raises the win rate to 71.0% and accounts for about three quarters of the total utility gain. The allocation of responses includes all ten models we consider, which makes routing worthwhile and highlights that measured intelligence is not the only relevant model characteristic. Next, we investigate the feasibility of learning individual preferences using repeat interactions. We find that after forty feedback rounds, we can close 72% of the gap to the optimal router. We also consider the efficacy of verbalizing estimated tastes. This approach yields a 70.4% predicted preference for the intended user versus a non-verbalized prompt, and steering the baseline model alone gains about as much as the optimal router. Lastly, because these routers select among generated responses, we evaluate routers that choose a model before generation: the prompt alone is nearly worthless, but a taste vector learned from forty past choices combined with predicted response characteristics recovers 35% of the ex-post oracle gain without generating any candidate response.

1 Introduction

A common view of progress in AI is that it is driven primarily by scaling training data and compute. Scaling has proven fruitful, with models improving over time on a variety of benchmarks. An implicit assumption behind scaling progress is that there is a primary

dimension of intelligence that explains most of the value of the model. Yet LLM responses also differ in length, tone, format, and other dimensions not captured by benchmark intelligence. If consumers differ in their preferences for these characteristics across different tasks, then no single model will be best for everyone. In this paper, we document large gains in utility from using the prompt and user’s preferences to route requests across models. To do so, we propose an empirical framework for measuring the gains from a variety of routing approaches.

We begin by estimating a random-coefficients discrete-choice model in which utility depends on the judged accuracy of a response, model identity and capabilities, response style, the prompt, and a user’s latent tastes. The estimation uses preference data from Arena, a platform that presents users with anonymized responses from two randomly assigned models and asks which they prefer (Chiang et al. 2024). Every response is scored for factual accuracy by an LLM judge, and accuracy enters utility in three bins, treated symmetrically with every other response characteristic: a mean effect, an interaction with the prompt, and a user-specific taste. Accuracy is the biggest driver of preference, though with diminishing returns: a response that is partly right compared to a wrong answer matters about 1.7 times as much as a response that is mostly or fully right compared to a partly right one. How much accuracy matters depends strongly on the prompt, and users differ widely in how much they value a partly right answer, while agreeing on the value of a fully right one. Style preferences are not displaced by accuracy, so they are not a proxy for correctness; the two act as separate factors of preference. Holding accuracy fixed, we document heterogeneous preferences for a variety of characteristics (e.g., tables, response length) and interactions between these characteristics and prompt embeddings. These heterogeneous preferences affect which model is most preferred. If each user must be assigned a single model, 56% have a personal optimum different from the mean-taste winner. No model dominates the ten-model candidate set: under personalized assignment, the leading model accounts for only 25.8% of user-prompt choices, and the top two models together receive 43.8%, leaving more than half to eight other models.

These differences in preferences and model responses generate substantial gains from routing. Relative to always using the best single model, Gemini 2.5 Pro, a prompt-level router raises mean predicted utility and attains a 53.8% model-implied head-to-head win rate. Giving the router oracle knowledge of each user’s taste vector further increases utility gains and the win rate to 71.0%. Personalization accounts for about three quarters of the total utility gain. This benefit comes from frequently selecting models that have lower benchmark intelligence on average than the prompt-only router’s selections.

Next, we consider whether the oracle benchmark can be approached in practice by learning preferences through repeated elicitation of pairwise preferences. We simulate 40 rounds of pairwise feedback, and use a Bayesian approach to learn the preference vector.

This approach closes 72% of the utility gap between an unpersonalized router and the oracle. This exercise is closely related to recent methods that represent users by latent variables or ideal points in a shared preference space (Poddar et al. 2024; Chen et al. 2025), as well as Shenfeld et al. (2025), who factorize user-specific rewards into shared base functions and use active learning to infer a new user’s weights from a small number of pairwise comparisons. We also test a simpler intervention that requires no model fine-tuning: a natural-language preamble instructs Gemini 2.5 Pro to match a user’s estimated tastes. The resulting responses are preferred by the intended user over the unsteered response with predicted probability 70.4%, and the same holds for every model we steer. Measured against the same baseline on the same prompts and users, steering Gemini alone gains 1.36 logits (win rate 69.2%), about as much as the oracle router on the same nine-model menu (1.25) and far more than the prompt router, and steering the best model for each user and prompt gains 2.30 logits (win rate 80.0%).

These gains do not come for free. Our main router operates *ex post*: it generates one response from each of ten models and then selects among them, increasing inference costs and exposing the comparison to response-level sampling noise. We evaluate *ex ante* routers that choose a model using only information available before generation, thus saving the cost of prompting all ten models before routing. We benchmark these against the same baseline, always using Gemini 2.5 Pro, on held-out prompts and users, where the *ex post* oracle gains 1.30 logits and the *ex post* prompt router 0.30. In one approach, we predict each model’s response characteristics, including its accuracy bin, from the prompt embedding and score them through the choice model. This prompt-only *ex ante* router gains only 0.03 logits: response characteristics are hard to predict before generation. In another approach, we built a router that uses a taste vector learned from 40 of the user’s past pairwise choices to pick one best model without generating responses from all models; it gains 0.32 logits and a 55.3% win rate, 24% of the oracle gain. Combining the two approaches raises the gain to 0.45 logits, 35% of the oracle gain and a 57.2% win rate, which exceeds the *ex post* prompt router without generating any candidate response. A replicated sampling audit shows why prompt-only prediction fails: on a fixed prompt, re-sampling the same model moves its utility by nearly as much as the prompt-specific edge that a router exploits. The routing exercises are structural, model-scored counterfactuals rather than estimates of the performance of a production router.

The closest paper to ours is Caoui (2025), who uses LMArena choices to estimate demand for differentiated LLMs and decomposes changes in consumer surplus into improvements in intelligence and other model attributes. We use a similar demand framework but shift the counterfactual from measuring market-wide quality growth to assigning responses across models and users. Our contribution is to quantify separately the value of prompt-level routing and user-level personalization in a choice model that holds response-level

accuracy fixed, and to evaluate two mechanisms for learning and acting on individual tastes. The paper also complements work documenting pluralistic preferences in human feedback (Kirk et al. 2024) and divergence between aggregate and individual LLM rankings (Garbacea, Wang, and Tan 2026). Our findings are also consistent with Demirer, Fradkin, and Tadelis (2026), who document horizontal and vertical differentiation in the market for LLM inference and show that no single model dominates across use cases. More broadly, our results provide empirical content for models in which user heterogeneity and customization shape the product design and pricing of LLM services (Bergemann, Bonatti, and Smolin 2025).

The paper proceeds as follows. Section 2 describes the data, the response attributes, the accuracy judge, and the choice model. Section 3 reports the estimates. Section 4 measures the value of prompt routing and personalization and the feasibility of learning tastes from feedback. Section 5 personalizes a single model with language. Section 6 evaluates ex-ante routers, and Section 7 audits response sampling noise. Section 8 discusses limitations and implications, and Section 9 concludes. The appendices give the data construction, the full coefficient tables and identification diagnostics, and the details of every simulation.

2 Data and empirical framework

Arena, originally introduced as Chatbot Arena and also known as LMArena, is a widely used online platform for evaluating large language models through human preference comparisons. Its public model rankings are frequently cited by leading model developers and companies (Chiang et al. 2024). Users submit their own prompts and receive side-by-side responses from two randomly assigned models, whose identities remain hidden until after voting. Users can select either response, declare a tie, or judge both responses to be bad. These comparisons, called “battles,” are aggregated into a public leaderboard. The platform therefore collects evaluations of responses to user-chosen tasks, encompassing both substantive quality and presentation. We use the underlying battle-level choices to estimate preferences over model and response characteristics.

2.1 Arena choices and response attributes

We pool the public `arena-human-preference-100k` and `arena-human-preference-140k` releases, covering June–August 2024 and April–July 2025. The English corpus has 128,850 battles (257,700 responses); after dropping incomplete observations and the 18 battles in which a response could not be judged for accuracy, 128,366 battles remain. The seeded 90/10 split contains 115,440 training and 12,926 test battles. The outcomes are A win (33.6%), B win (34.5%), tie (17.2%), and both bad (14.7%). We randomize ordinary ties between A and B using a fixed seed and treat “both bad” as an outside option.

Table 1: Estimation data. Counts are battles; outcome shares use the 128,850-battle English corpus before the complete-feature and judged-accuracy restrictions.

Sample	Battles	Outcome	Share
English corpus	128,850	A wins	33.6%
Complete estimation sample	128,366	B wins	34.5%
Training split	115,440	Tie	17.2%
Held-out split	12,926	Both bad	14.7%

Each response has thirteen features: token length; counts of ordered lists, unordered lists, headers, and bold spans; indicators for tables and emoji; Flesch reading ease and the SMOG index; positive and negative sentiment; and assertive and tentative hedging. Deterministic parsers construct all but the final four indicators, which `openai/gpt-oss-20b`, called through OpenRouter at temperature zero, classifies. Continuous features are standardized using training-sample moments. Prompts are embedded with OpenAI’s `text-embedding-3-large` and reduced to three partial-least-squares (PLS) scores, allowing the value of an attribute to change with the request; the projection is fit on training battles only and applied to held-out and simulation prompts. Appendix A provides the full construction.

Each model has three attributes that are constant across prompts: a benchmark intelligence index, context window, and a reasoning indicator. We measure intelligence with the Artificial Analysis Intelligence Index, a composite of benchmark scores reported on a nominal 0–100 scale, and enter it in units of ten points. We use this index because it covers all 106 models in the estimation sample;¹ The context window is measured in millions of tokens, and the reasoning indicator equals one for models that use extended chain-of-thought reasoning before responding.

2.2 Judged accuracy

Every response also carries a judged factual-accuracy score. A GLM 5.2 judge (`z-ai/glm-5.2` through OpenRouter) sees the prompt and one response and returns a score from 0 to 10 with a short rationale; 257,682 Arena responses were scored and 18 calls failed. On a validation sample of 50 battles (96 responses) also scored by a Claude Opus 4.8 judge, the two judges’ scores correlate at 0.80 and induce the same within-battle ordering in 95% of the 21 battles in which both are decisive. Scores enter the model as three bins: low (0–3; 14.8% of responses), mid (4–6; 18.0%), and high (7–10; 67.2%). The binning lets the value of accuracy be nonlinear and avoids treating the judge’s scale as cardinal. The same judge scores every response used downstream: the ten-model routing panel, the replicated sampling audit, and the verbally personalized generations, so no counterfactual is ever

¹The Epoch Capabilities Index predicts judged accuracy (Section 2.2) somewhat better at the model level (R^2 of 0.70 against 0.57) but is available for only 73 of them.

scored without the accuracy term. Appendix B gives the judge details.

2.3 Choice model

Let i index users, p index prompts, and m index language models. In each battle, a user compares two responses to the same prompt and either selects one or rejects both. The systematic utility from model m 's response to prompt p is

$$V_{ipm} = \left(\gamma_0 + \Gamma h_p + \text{diag}(\sigma^A) \nu_i^A \right)' d_{pm} + \left(\beta_0^R + B^R h_p + \text{diag}(\sigma^R) \nu_i^R \right)' x_{pm}^R \\ + \left(B^M h_p + \text{diag}(\sigma^M) \nu_i^M \right)' x_m^M + \delta_m + \alpha \mathbf{1}\{\text{position B}\}, \quad (1)$$

where the superscripts A , R , and M distinguish accuracy, response, and model characteristics. The vector d_{pm} contains the two accuracy-bin indicators (mid and high, with low as the reference). The vector x_{pm}^R contains the thirteen response attributes described above, including length, formatting, readability, sentiment, and hedging. The vector x_m^M contains three model characteristics: the benchmark intelligence index, context window, and reasoning indicator. These are constant across prompts for a given model. The vector h_p contains the three PLS scores constructed from the prompt embedding.

Accuracy is treated symmetrically with the other response characteristics. The coefficient vector γ_0 gives the mean value of a mid and a high response relative to a low one at $h_p = 0$; the 2×3 matrix Γ lets that value change with the prompt; and σ^A lets users differ in it. The coefficient vector β_0^R gives mean preferences for response attributes at $h_p = 0$. The matrices B^R and B^M , of dimensions 13×3 and 3×3 , respectively, shift mean preferences with the prompt: $\beta_0^R + B^R h_p$ is the population mean response-coefficient vector conditional on prompt p , while $B^M h_p$ captures prompt-dependent shifts in preferences for model characteristics. Baseline differences across models enter through δ_m ; the specification does not include a separate vector of common mean coefficients on x_m^M .

Unobserved preference heterogeneity enters through $\nu_i = ((\nu_i^A)', (\nu_i^R)', (\nu_i^M)')' \sim N(0, I_{18})$, where ν_i^A , ν_i^R , and ν_i^M contain two, thirteen, and three independent standard-normal taste components. The vectors σ^A , σ^R , and σ^M give the standard deviations of the random coefficients, and $\text{diag}(\sigma)$ denotes a diagonal matrix with σ on its diagonal. For a given prompt, a larger element of σ implies greater dispersion across users in the corresponding marginal utility. The same taste vector applies to both responses in a battle. Because individuals are not tracked across battles, we estimate population heterogeneity rather than preferences for identifiable users.

The fixed effect δ_m captures persistent differences in utility across models after accounting for judged accuracy, observed response attributes, and prompt interactions. There are 74 fixed effects: one for each named model from the seven largest providers and a pooled effect for all other models. Finally, $\mathbf{1}\{\text{position B}\}$ equals one if the response appears in

position B and zero otherwise; α captures a common position effect.

To obtain choice probabilities, let $j \in \{A, B, 0\}$ index the two displayed responses and the outside option, where $j = 0$ denotes a “both bad” vote. Total utility is $U_{ipj} = V_{ipj} + \varepsilon_{ipj}$, with V_{ipA} and V_{ipB} given by Equation (1) for the respective models. We normalize systematic outside-option utility to $V_{ip0} = 0$ and assume that the ε_{ipj} are independent standard type-I extreme-value shocks, independent of ν_i . These assumptions fix the location and scale of utility. Integrating the conditional logit probability over the taste distribution gives

$$P_{ip}(j) = \int \frac{\exp\{V_{ipj}(\nu)\}}{1 + \exp\{V_{ipA}(\nu)\} + \exp\{V_{ipB}(\nu)\}} \phi_{18}(\nu) d\nu, \quad (2)$$

where ϕ_{18} is the standard multivariate-normal density in eighteen dimensions; the numerator is one for the outside option. We estimate the model’s 162 parameters by simulated maximum likelihood, approximating the integral with 256 Sobol/antithetic draws (McFadden and Train 2000). Every downstream number is computed from Equation (1) through one shared scoring path, so the routers, the learning simulation, the verbal experiment, and the sampling audit are scored identically by construction.

2.4 Identification

Identification of user-level dispersion comes from how the observed choice probability changes as response-feature gaps vary across battles. Because a person is not tracked longitudinally, the normal mixing distribution is a population model, not a claim that we have recovered any observed user’s vector. To assess parameter recovery, we generate synthetic choices from the model using the reported estimates as the true parameter vector (80,000 battles, 80 models, 100 replications) and re-estimate it. Mean response coefficients, the accuracy gains, and fixed effects recover closely (fixed-effect correlation 0.99 in every replication, accuracy gains within 2%), and the large spreads that carry the heterogeneity interpretation (length 1.07, emoji 1.43, table 1.45, tentative hedging 0.70) are reproduced within 3% on average. True taste spreads below roughly 0.1 are not recovered: they return at the 0.02 optimization floor in 40 to 54% of replications and inflated by a multiple in the rest, and the positive-tone spread (0.41) is understated by 13%. This pattern motivates our focus on length, table, emoji, and hedging heterogeneity; small spreads do not support substantive claims.

3 Preference estimates

3.1 Fit and the value of accuracy

Table 2 shows that accuracy is first-order. A standard logit with the two accuracy bins and the style features predicts held-out battles better than a standard logit with model

Table 2: Choice-model fit and selected estimates. NLL is negative log-likelihood per held-out battle; lower is better. Accuracy bins are low (0–3, reference), mid (4–6), and high (7–10). Hessian standard errors are in parentheses.

Specification / parameter	Estimate	Test NLL
Standard logit, model fixed effects only (75 parameters)		0.9884
Standard logit with style and accuracy bins (90 parameters)		0.9483
Mixed logit, common accuracy coefficients (154 parameters)		0.9423
Mixed logit, accuracy symmetric with style (162 parameters)		0.9345
Accuracy: mid vs. low, mean	1.100	(0.026)
Accuracy: high vs. low, mean	1.761	(0.026)
Accuracy: mid vs. low, user-level spread	1.592	(0.068)
Accuracy: high vs. low, user-level spread	0.027	(0.081)
Style: response length (per SD)	0.491	(0.028)
Style: table indicator	0.450	(0.053)
Style: emoji indicator	0.196	(0.030)

fixed effects alone (0.9483 against 0.9884 per battle), and a mixed logit in which the bins enter with common coefficients reduces test NLL per battle to 0.9423. Treating accuracy symmetrically with the other response characteristics, with prompt interactions and a user-level spread, reduces it further to 0.9345: a likelihood-ratio statistic of 1,473 on eight degrees of freedom. The effect is sharply concave. The low-to-mid gain is 1.100 logits, whereas the incremental mid-to-high gain is $1.761 - 1.100 = 0.661$ (Figure 1), so a partly right answer against a wrong one matters about 1.7 times as much as a fully right answer against a partly right one. Two model-free comparisons make the magnitude concrete. Among 70,208 battles with a decided or randomized outcome in which judged scores differ, the response with the higher score wins 57.3% of the time; restricting to the 35,632 such battles whose responses fall in different accuracy bins raises that share to 63.1%. The structural estimates are larger because they compare otherwise similar responses and separate accuracy from identity and style. The concavity also cautions against treating a judge’s 0–10 scale as cardinal: moving an answer out of the clearly weak tail matters more than polishing a medium answer into the high bin.

How much accuracy matters depends strongly on the prompt. The interaction of the high bin with the first prompt score is 5.26 (standard error 0.16), and of the mid bin 3.27 (0.20), so the value of a correct answer is several times larger on the prompts at one end of the first PLS axis than at the other. Users also differ in how much they value a partly right answer: the user-level spread of the mid-bin gain is 1.59 against a mean of 1.10, which implies that a sizeable minority of users value a partly right answer above a fully right one, whereas the spread of the high-bin gain is at the optimization floor. Users disagree about how much a partly right answer is worth, and agree on the value of a fully right one.

A simpler demand specification tells the same story from the other direction. In a

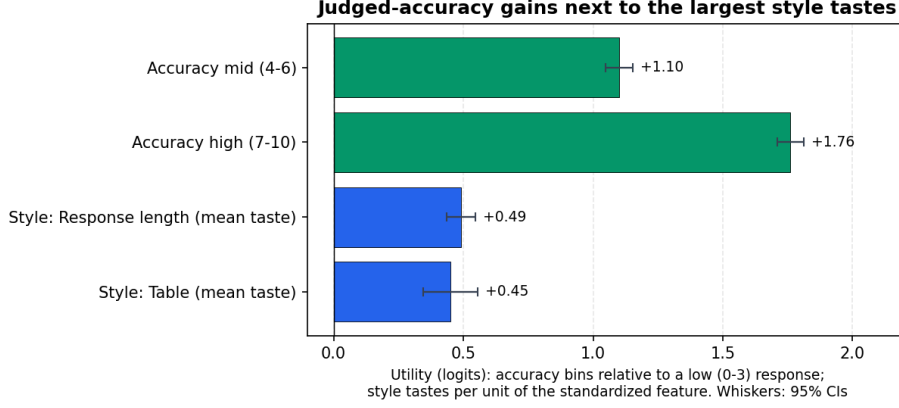


Figure 1: Judged-accuracy gains next to the two largest mean style tastes. Accuracy bins are relative to a low (0–3) response; style tastes are per unit of the standardized feature. Whiskers are 95% confidence intervals.

pairwise logit of decided battles on the difference in benchmark intelligence alone, ten index points are worth 0.215 logits; adding the model capabilities, the thirteen style differences, and the two accuracy-bin differences lowers the intelligence coefficient by 22.6% to 0.166 while the accuracy bins enter at 0.83 and 1.16 (Appendix C). Part of what a benchmark index predicts about choice is realized accuracy and style, not an intrinsic model premium.

3.2 Style heterogeneity holding accuracy fixed

Accuracy does not displace style. Figure 2 shows where heterogeneity is identified once correctness is held fixed. Tables have a mean coefficient of 0.450 and a user-level spread of 1.448; emoji, 0.196 and 1.432; and standardized length, 0.491 and 1.066. Tentative hedging has a negative mean coefficient (−0.132) but a sizeable spread (0.699), and positive tone a mean of 0.110 and a spread of 0.405. By contrast, list counts, bold text, and assertive hedging have spreads at or near the optimization floor. The estimates show that a few dimensions exhibit wide preference heterogeneity, and that for these characteristics the sign of a user’s coefficient often differs from the population mean. Vertical and horizontal differentiation coexist: once a response is reasonably accurate, further accuracy gains are worth less and style differences carry more weight.

Prompts shift these tastes. The interaction of length with the second and third prompt scores is large and precisely estimated (−3.40 and −1.82), so the same length is valued very differently across requests, and the table interactions (−1.26, −1.57, and 1.91) show that a table is a benefit on some prompts and a cost on others. Appendix C reports the full interaction block.

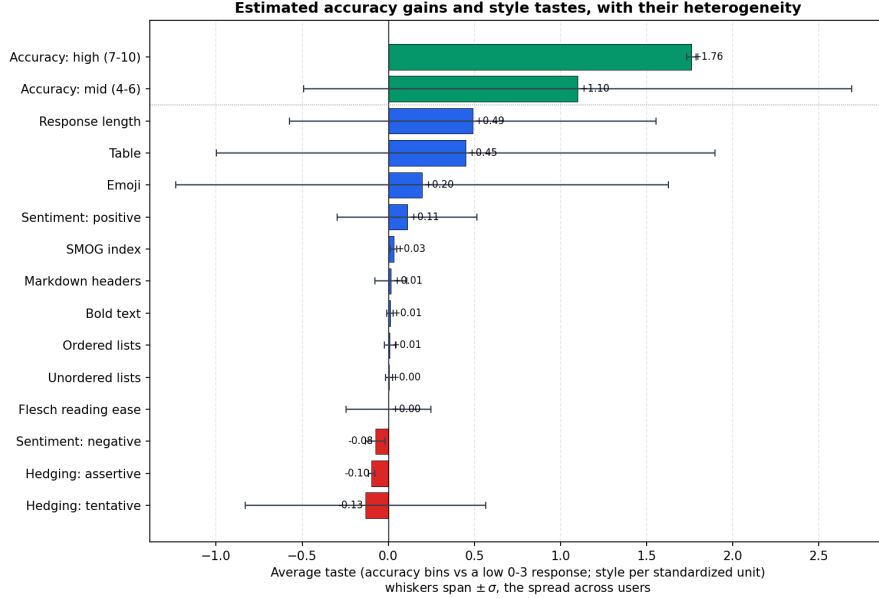


Figure 2: Population mean style coefficients, holding judged accuracy fixed (bars), and mean $\pm$ estimated user-level taste spread (whiskers). Large spreads are concentrated in length, tables, emoji, and hedging.

3.3 Model fixed effects and the second stage

Model fixed effects are the other large source of utility. They absorb persistent differences in capability, brand, default behavior, and any omitted response attribute correlated with identity, as well as average effects of model attributes that are constant across prompts, net of the accuracy and style already in the model (Figure 7 in the appendix plots all 74). A second-stage OLS regression of the 73 named fixed effects on model attributes associates a ten-point increase in the benchmark index with 0.160 logits ($t = 5.3$). It is important to note that these coefficients should not be interpreted as causal, since model attributes are correlated with unobserved model characteristics. For example, models with higher benchmark scores may also be more likely to be trained to return good chat responses. A particular concern for a causal interpretation is that intelligent models may be more likely to cater the characteristics of the response to the prompt. We therefore interpret these coefficients as suggestive of explanations for persistent across model differences. Appendix C repeats the second stage on the fixed effects of two plain logits: the association falls by about half once response characteristics enter, which is what one expects if part of what more capable models are rewarded for is more accurate and better formatted answers.

3.4 Differentiation in the candidate menu

The ten-model panel is not vertically ordered for all users. Under the oracle, Gemini 2.5 Pro and Grok 4 together receive 43.8% of user-prompt assignments, leaving more than half

Table 3: Oracle assignment shares over user-prompt cells. Users may switch models across prompts.

Model	Share	Model	Share
Gemini 2.5 Pro	25.83%	MiniMax M1	6.19%
Grok 4	17.95%	Claude Sonnet 4	5.85%
DeepSeek V3	12.48%	Kimi K2	4.62%
Qwen3 235B	11.04%	Llama 4 Maverick	2.77%
GPT-4.1	10.72%	GPT-4o	2.55%

to eight other models (Table 3). GPT-4o, although a strong and widely used model, is the oracle choice in only 2.5% of cells. These are counterfactual shares under the estimated taste distribution, not market shares; they isolate differentiation from prices, defaults, access, and switching costs.

This assignment object differs from asking which single model maximizes a user’s utility averaged over prompts. In that one-model-per-user exercise, Gemini 2.5 Pro is the personal best for 44% of users and Grok 4 for 16%, with DeepSeek V3, GPT-4.1, and Qwen3 235B each the personal best for about 12%; 56% of users have a personal best that differs from the mean-taste winner. Keeping the objects separate avoids a common but misleading comparison between a per-user share and a user-prompt share. A single model can be the modal personal choice without being the choice for everyone, and once assignment can also vary across prompts the menu becomes far more dispersed. The same dispersion appears by topic: the population pick share of the leading model, Gemini 2.5 Pro, is 17.4% overall, and it leads five of seven prompt topics, with Qwen3 235B leading mathematics and Grok 4 multimedia prompts; the leader is statistically distinguishable from the runner-up in every topic but multimedia (Appendix D).

4 The value of personalized routing

4.1 Policies and evaluation

The response panel contains 4,916 topic-stratified prompts with complete generations from ten models: GPT-4.1, GPT-4o, Claude Sonnet 4, Gemini 2.5 Pro, DeepSeek V3, Llama 4 Maverick, Grok 4, Qwen3 235B, Kimi K2, and MiniMax M1. Appendix D gives the exact API identifiers. Each prompt-model cell has one response, scored for style and judged for accuracy. We compare:

1. *Fixed*: always serve Gemini 2.5 Pro, the best single model in the panel.
2. *Prompt router*: serve $\arg \max_m V_{0pm}$, setting $\nu_i = 0$.
3. *Oracle personalized router*: serve $\arg \max_m V_{ipm}$ using the user’s true simulated ν_i .

Table 4: Model-scored welfare relative to always using Gemini 2.5 Pro. The final row is the incremental oracle gain over the prompt router.

Policy	Mean ΔU	Implied win rate
Prompt router	0.305	53.8%
Oracle personalized router	1.321	71.0%
Personalization: oracle – prompt	1.016	67.2%

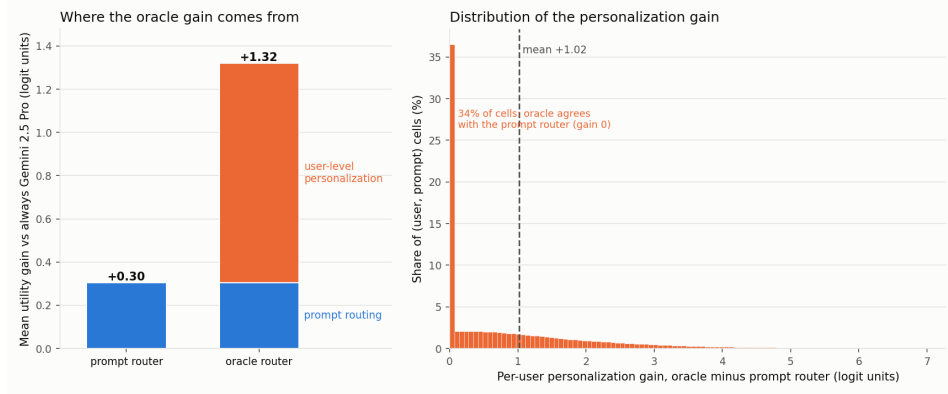


Figure 3: The model-scored gain from the oracle personalized router. Left: mean utility relative to always using Gemini 2.5 Pro decomposes into prompt routing and user-level personalization. Right: distribution of the incremental personalization gain. Both axes use logit utility units.

The counterfactual samples 1,000 users from $N(0, I)$ with a fixed seed. For policy r , utility gain is $\Delta U_{ipr} = V_{ip,r(p,i)} - V_{ip,\text{Gemini}}$. We report mean gain and the average logit probability $\text{logit}^{-1}(\Delta U)$ that the policy’s response wins a pairwise comparison. These are predictions of the estimated utility model; no new human votes are collected.

4.2 Prompt routing and the oracle personalization prize

The prompt router raises mean predicted utility by 0.305 logits and reaches a 53.8% model-implied win rate against Gemini 2.5 Pro (Table 4). The gain is modest because the baseline is strong: the prompt router serves Gemini itself on 53.6% of prompts, where the gain is zero by construction, and gains are positive in 28.3% of user-prompt cells. The oracle adds another 1.016 logits, about 3.3 times the prompt-routing gain, and raises the total gain to 1.321. Because the prompt router’s choice is in the oracle’s choice set, the incremental gain is nonnegative by construction; it is strictly positive in 65.5% of cells. Personalization is therefore about three quarters of the total gain from routing.

The distribution of choices also shows differentiation. When one model must be assigned to a user across all prompts, Gemini 2.5 Pro is the personal best for 44% of users and six models exceed 2%; even the modal choice leaves more than half the population better

served elsewhere. When users may also switch by prompt, no model exceeds one quarter of oracle user-prompt assignments. The oracle’s gains are therefore not merely a relabeling of one universally superior model.

The measured gains mix model fixed effects, judged accuracy, style, prompt interactions, and taste matching. A decomposition of the 0.30 prompt-router gain assigns -0.13 logits to fixed effects, 0.12 to judged accuracy, 0.27 to mean style, and 0.04 to prompt interactions: the router gives up fixed-effect utility relative to Gemini and more than recovers it through accuracy and style. The oracle adds 1.02 from taste matching, part of which is each user’s own taste for a partly right answer. Only part of the fixed-effect component is statistically associated with benchmark intelligence, so converting the total into benchmark points is not meaningful. The direct capability index of selected models makes the same point: it averages 67.9 for the prompt router but falls to 62.5 under the oracle, which willingly trades vertical quality for taste fit.

The two personalization margins operate differently. Prompt routing exploits predictable changes in average tastes, Bh_p and Γh_p , as well as realized response features. It can send a coding prompt and a writing prompt to different models without knowing who submitted either request. User personalization instead changes the coefficient vector applied to the same candidate set. Its value is greatest when models span the attributes on which σ is large and when a user’s taste draw is far from zero. The oracle gain would shrink if suppliers converged to the same response style, even if their average capability stayed unchanged.

Ablations support this interpretation but also reveal the importance of persistent model quality. Setting all model fixed effects to zero changes 48.0% of mean-user prompt choices on the 4,916-prompt panel. Setting the entire block of mean response-style coefficients to zero changes 52.4% and concentrates nearly all assignments on xAI and Google. Removing the accuracy gains changes 10.9% : the candidate models are close in judged accuracy on most prompts, so accuracy separates responses less often than style does within this menu. Among individual style terms, length changes 22.0% of choices when its mean coefficient is removed; tables have a comparable mean coefficient but affect fewer choices because the feature is binary and rare. These exercises are descriptive perturbations of the fitted objective, not causal estimates of how a provider would change its response if a feature were mandated.

The outside option also matters for interpreting the scale. Utility is normalized relative to “both bad,” not to the least-preferred model in the candidate menu. A gain of one logit therefore changes both relative model choice and acceptability. Our reported head-to-head rate uses $\text{logit}^{-1}(\Delta U)$ and conditions the comparison on the two policy responses; it should not be confused with the probability that either response clears the outside option. Reporting both raw utility and the pairwise transform keeps this distinction visible.

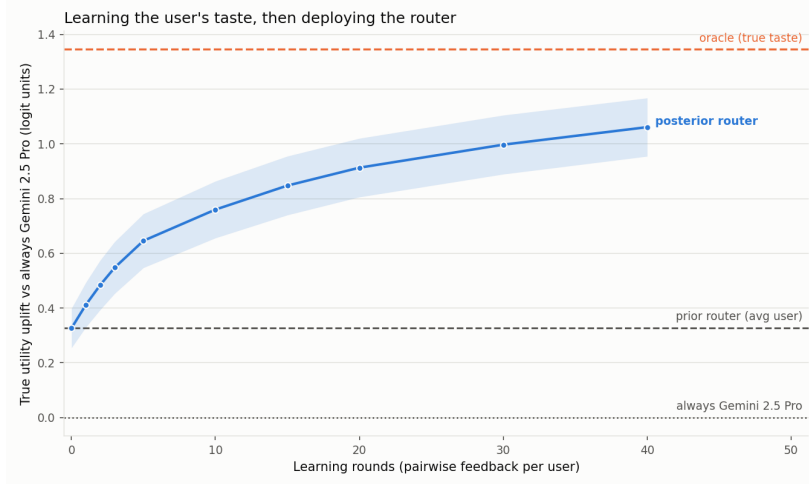


Figure 4: Online preference learning. Mean model-scored uplift over Gemini 2.5 Pro rises from 0.327 logits at the prior to 1.061 after 40 comparisons, against an oracle of 1.345 on this evaluation sample. Bands are 95% intervals over users.

4.3 Learning tastes from pairwise feedback

The oracle assumes away the cold-start problem. We therefore simulate a Bayesian learner for 200 new users, evaluated on 800 held-out prompts. In each feedback round the current posterior-best response is paired with a random challenger; position is randomized and the simulated user chooses A, B, or both bad according to Equation (1). We compute a maximum-a-posteriori estimate of ν_i under its $N(0, I)$ prior after each round, then route with the posterior taste estimate.

The prior router gains 0.327 logits over Gemini 2.5 Pro. After 10 comparisons the posterior router gains 0.760 and closes 43% of the oracle gap; after 40, it gains 1.061, reaches a 66.0% implied win rate, and closes 72% (Figure 4; the full curve is in Appendix D). This simulation establishes feasibility under correct model specification. It does not measure how real users respond to the proposed queries or whether tastes remain stable over time.

5 Personalizing one model with language

Routing requires access to several models and, in our ex-post implementation, a response from every candidate. A lower-infrastructure alternative is to change how one model responds. We sample 100 taste profiles from the estimated distribution, stratified by the length coefficient. A deterministic rule converts each profile’s largest style-taste deviations into at most four natural-language instructions, such as preferences for concise answers, tables, emoji, or assertive language; the profile’s taste for accuracy is never verbalized, since nobody asks for a less accurate answer, but it enters the scoring of that user’s responses. For 100 topic-stratified prompts, Gemini 2.5 Pro (`google/gemini-2.5-pro`

Table 5: Verbal personalization of Gemini 2.5 Pro. Utility differences compare a steered response with the unsteered response for the same prompt.

Evaluation	Mean ΔU
Intended user’s taste (diagonal)	1.448
Average user’s taste	0.056
Different user’s taste (off-diagonal)	−0.014

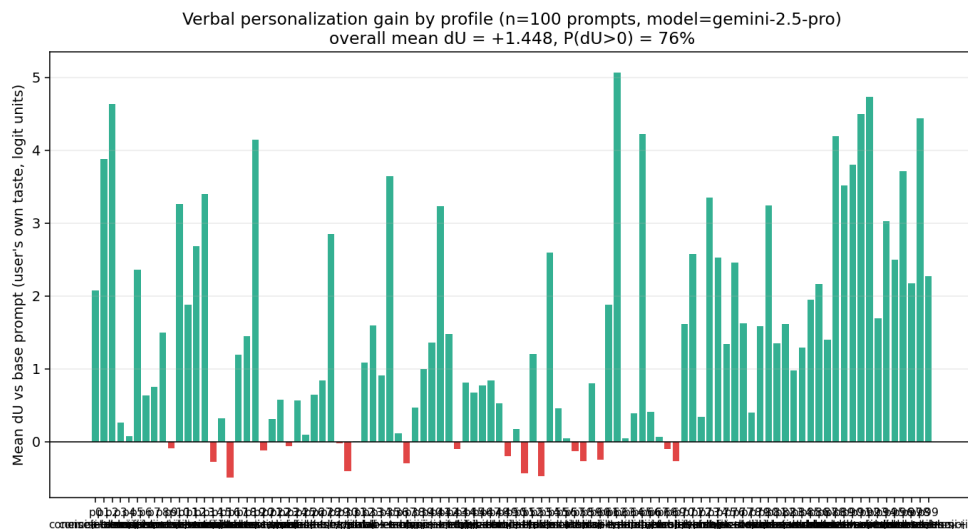


Figure 5: Mean verbal-personalization gain by taste profile for Gemini 2.5 Pro, ordered from the most concise to the most verbose length taste. Each point averages over 100 prompts. Gains are larger toward both tails but also reflect the table, emoji, and hedging instructions bundled into each profile.

through OpenRouter) produces one unsteered response and one response for every profile under the same generation policy as the routing panel (10,000 steered responses; temperature 0.7, 4,000-token cap). Every response is judged for accuracy, so the utility difference includes the accuracy term.

The intended user’s mean gain is 1.448 logits, with a prompt-clustered standard error of 0.053 ($t = 27.4$); 75.8% of cells improve, and the mean pairwise transform is 70.4%. The two placebo evaluations in Table 5 are central. Under the average user’s tastes the same feature changes are worth 0.056, and in the full 100×100 cross-matrix a response written for somebody else is worth -0.014 . Thus the diagonal gain is not explained by verbal preambles making all responses generically better under the model.

Tables contribute 0.519 logits (35.9% of the total), length 0.512 (35.4%), and emoji 0.401 (27.7%); judged accuracy contributes only 0.020, so the preamble changes how a response looks rather than whether it is right; other style features net to -0.005 . Gains are U-shaped in taste extremity: profiles near the center have little to personalize, while

Table 6: Verbal personalization across models. Columns 2 to 4 compare each model’s steered response with its own unsteered response under the intended user’s tastes (100 profiles $\times$ 100 prompts). Columns 5 and 6 compare every policy with always serving Gemini 2.5 Pro’s unsteered response on the 9,745 user-prompt cells in which all nine steered responses exist; the routing rows use the nine-model menu that can be steered and the ten-model panel of Section 4.

Policy	Steered vs. own unsteered			vs. always Gemini	
	Mean ΔU	Win rate	Placebo ΔU	Mean ΔU	Win rate
Verbal, Gemini 2.5 Pro	1.448	70.4%	−0.014	1.363	69.2%
Verbal, Qwen3 235B	1.398	68.8%	−0.017	0.529	55.8%
Verbal, Claude Sonnet 4	1.315	69.0%	0.035	0.728	59.0%
Verbal, GPT-4.1	1.249	68.3%	0.037	0.746	59.4%
Verbal, Kimi K2	1.229	67.6%	−0.097	0.462	55.4%
Verbal, MiniMax M1	1.188	66.9%	−0.021	0.245	51.8%
Verbal, GPT-4o	1.164	67.2%	0.022	0.343	53.8%
Verbal, Llama 4 Maverick	1.071	66.6%	−0.043	0.109	50.5%
Verbal, DeepSeek V3	0.967	64.9%	−0.121	0.585	57.8%
Prompt router, nine models				0.224	52.7%
Oracle router, nine models				1.245	70.1%
Prompt router, ten models				0.269	53.4%
Oracle router, ten models				1.344	71.7%
Verbal, best model per user and prompt				2.300	80.0%
Oracle over panel and steered responses				2.528	83.1%

strongly concise, verbose, pro-table, or anti-table profiles have more (Figure 5).

The result is not specific to Gemini. Table 6 repeats the experiment for every model in the live menu. Steering raises the intended user’s utility by 0.97 to 1.45 logits on every model, the placebo is within 0.12 of zero on every model, and the predicted preference for the steered response ranges from 64.9% to 70.4%. The table also puts verbalization and routing on the same scale, relative to always serving Gemini 2.5 Pro’s unsteered response on the same users and prompts. Steering Gemini alone gains 1.36 logits, about as much as the oracle router over the nine steerable models (1.25) and the ten-model panel (1.34), and far more than the prompt router (0.22). Steering a weaker model is worth less than routing because the user gives up Gemini’s fixed effect, but choosing the best steered model for each user and prompt gains 2.30 logits, nearly twice the oracle router, and adding the steered responses to the oracle’s menu raises its gain from 1.34 to 2.53. Verbalization and routing are complements: one changes how a model responds, the other which model responds. This experiment uses model-scored utility. It is evidence that a capable LLM can translate coarse preference instructions into targeted feature changes, not a substitute for a blinded human evaluation.

6 Ex-ante routing: what is reachable before generation

The routers of Section 4 are ex post. The prompt router reads the realized characteristics of every candidate response, and the oracle reads those characteristics and the user’s true taste. Both bound the prize without saying whether any of it is reachable by a system that must pick a model before anything is generated. This section evaluates such ex-ante routers on the same panel.

6.1 Design

Two sources of information are available before generation: the user’s taste, known or learned from past choices, and the prompt. We evaluate three families of routers built from them, each choosing one model with no candidate response in hand.

- *Taste to model (A)*. One model per user, for every prompt. A0 uses the user’s true taste and picks the model with the highest expected utility in closed form; it bounds this family. A1 infers the taste by maximum a posteriori from k past pairwise choices and then applies the same rule. A2 skips the taste vector and trains a multinomial logistic classifier from the choice history straight to a model identifier, which tests whether routing the prediction through the structural model earns its keep.
- *Prompt to utility (B)*. One model per prompt, for every user. B1 and B2 predict each model’s utility from the prompt embedding with ridge regression and a dense network. B3 and B4 instead predict the fifteen response characteristics the utility function reads (the thirteen style features and the two accuracy-bin indicators) and score them through the estimated choice model. Because utility is linear in characteristics, the expected utility of a model on a prompt equals the utility of its expected characteristics, so no simulation over response draws is needed.
- *Combined (C)*. C1 weights B3’s predicted characteristics by the user’s true taste; C2 uses a taste learned from k past choices. These are the only rules that use both sources.

Reference points share the scale: R0 always serves the best single fixed model on the learning split, Gemini 2.5 Pro; R1 is the ex-post oracle with the true taste; R2 is the ex-post average-user router of Section 4, the ceiling for family B.

Nothing is scored on data it was fitted to. Prompts split 40/60 into a learning set (1,966) and an evaluation set (2,950). Per-user assignments and choice histories use only learning prompts, and every reported utility is on evaluation prompts. Family B’s predictions are cross-fitted over five prompt folds, so no prompt’s own realized responses inform its own prediction. Of 4,000 simulated users, half train A2’s classifier and every variant is reported on the other half. A user’s k past choices are passive: random model pairs on learning

Table 7: Ex-ante routers on held-out prompts and users. Gains are mean utility over always serving Gemini 2.5 Pro (R0), in logits, with 95% intervals over users; win rate is the pairwise transform against R0. k is the number of past pairwise choices observed per user.

Router	Information used	Gain	95% CI	% of R1	Win rate
R1 ex post oracle	realized responses, true taste	1.304	[1.272, 1.337]	100	70.7%
R2 ex post prompt router	realized responses	0.297	[0.274, 0.319]	22.8	53.8%
A0 true taste, one model per user	true taste	0.424	[0.396, 0.452]	32.5	57.1%
A1 learned taste, $k = 10$	10 past choices	0.166	[0.139, 0.193]	12.8	52.8%
A1 learned taste, $k = 40$	40 past choices	0.315	[0.287, 0.342]	24.1	55.3%
A2 classifier, $k = 40$	40 past choices	0.221	[0.191, 0.251]	17.0	53.8%
B1 ridge to utility	prompt embedding	0.022	[0.013, 0.030]	1.7	50.2%
B3 ridge to characteristics	prompt embedding	0.033	[0.024, 0.041]	2.5	50.3%
B4 network to characteristics	prompt embedding	0.016	[0.006, 0.026]	1.2	50.1%
C1 true taste + B3	true taste, prompt embedding	0.583	[0.554, 0.612]	44.8	59.4%
C2 learned taste + B3, $k = 10$	10 past choices, prompt embedding	0.277	[0.250, 0.304]	21.2	54.3%
C2 learned taste + B3, $k = 40$	40 past choices, prompt embedding	0.454	[0.425, 0.483]	34.8	57.2%

prompts, answered through the same trinomial logit as the estimation model, including the outside option and the position dummy. This asks what can be read off ordinary usage; the active case, where the router chooses what to show next, is Section 4.3. Whatever a rule picks, realized utility is read from the user’s true taste through the shared scoring path. Appendix E lists every predictor.

6.2 Results

Table 7 and Figure 6 give the results. Against always serving Gemini 2.5 Pro, the ex-post oracle gains 1.304 logits and the ex-post prompt router 0.297. Prompt-only ex-ante routing recovers little of the latter: the best prompt-only rule, B3, gains 0.033 logits, 11% of the prompt-router ceiling, and the dense network does no better. The reason is not that utility is unpredictable. Ridge regression explains 25% of the variance of utility across prompt-model cells, but most of that variance is the model fixed effect, which every predictor recovers immediately and which cannot change a pick. After sweeping out each model’s mean across prompts and each prompt’s mean across models, the remaining prompt-by-model interaction is the only variation an argmax can respond to, and there the out-of-fold R^2 is 0.11 to 0.14. Response characteristics are hard to predict before generation, and Section 7 shows that a large part of the realized interaction is sampling noise that no prompt-side predictor can see.

Taste is more useful. Knowing the true taste and assigning one model per user is worth 0.424 logits, 32.5% of the oracle and more than the ex-post prompt router, without any response in hand. A taste learned from passive choices approaches this bound as the history grows: A1 gains 0.093 logits after 5 choices, 0.166 after 10, and 0.315 after 40, which is 74% of the A0 bound, with the recovered taste vector correlating 0.47 with the true one. Routing the history through the structural model matters: at every k the classifier A2,

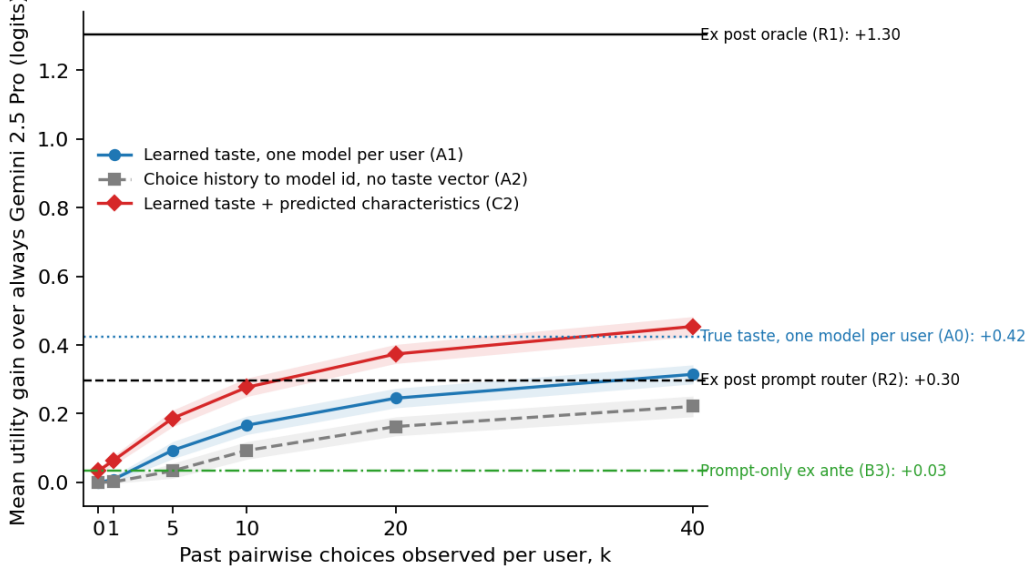


Figure 6: Ex-ante routing gains over always serving Gemini 2.5 Pro as a function of the number of past pairwise choices observed per user. Bands are 95% intervals over users. Horizontal lines are the ex-post oracle (R1), the ex-post prompt router (R2), the true-taste one-model-per-user bound (A0), and the prompt-only ex-ante router (B3).

which predicts a model identifier directly, trails A1, and at $k = 40$ it gains 0.221 against A1’s 0.315.

The two sources are complements rather than rivals. B3’s predicted characteristics are worth 0.033 on their own, yet combined with a learned taste they lift C2 at $k = 40$ to 0.454 logits, 34.8% of the oracle gain, with a 57.2% win rate against always serving Gemini 2.5 Pro; with the true taste, C1 reaches 0.583. C2 at $k = 40$ exceeds the ex-post prompt router (0.297) without generating a single candidate response, and it exceeds the A0 bound because the prompt-side prediction lets the per-user assignment vary across prompts. C2 is the deployable rule and the best-performing ex-ante router we evaluated. The gap that remains to the ex-post oracle, 0.85 logits, is the value of seeing the realized responses, which is exactly the quantity the sampling audit of the next section measures from the other side.

7 Response sampling noise

An ex-post router selects among one draw per model. A fully replicated auxiliary grid measures how much of what it selects on is repeatable. The grid contains 497 topic-stratified prompts, one tenth of the routing panel and the same prompts for every model, the nine candidate models that can still be queried, and five fresh temperature-0.7 responses per model-prompt cell drawn under the routing panel’s generation policy (22,334 scored

responses, each judged for accuracy; Grok 4 is excluded because its endpoint was retired before the replicates were drawn). The random-effects decomposition

$$U_{mpr} = \mu + \alpha_m + \beta_p + (\alpha\beta)_{mp} + \epsilon_{mpr}$$

assigns 16.6% of the variance of V_{0pm} to model, 36.3% to prompt, 27.2% to the model-prompt interaction, and 19.8% to within-cell resampling; the within-cell noise includes re-draws that land in different accuracy bins. In utility units, re-sampling the same model on the same prompt moves its utility by about 0.40 logits (one standard deviation), while the prompt-specific model edge that a router exploits has a standard deviation of 0.46 logits. Within a prompt, 72.8% of the utility spread across responses is between models and 27.2% is sampling noise from re-running the same model; models are statistically separable above the noise on 478 of the 497 prompts. Using one draw per cell selects the model with the largest five-draw cell mean 77.2% of the time (standard error 1.1 percentage points over prompts); the cell intraclass correlation is 0.80. Appendix F gives the decomposition by topic and by model.

Two conclusions follow. First, the ex-post router’s gains are computed on realized responses and therefore include the value of having drawn a good response, which a compute-matched policy that samples repeatedly from one model could partly replicate; this audit does not estimate that comparison, so we use it only to bound the reliability of one-draw routing. Second, the failure of prompt-only ex-ante routing in Section 6 has a structural reason: the routable prompt-by-model interaction is only somewhat larger than the draw-to-draw noise it has to be read through, and no predictor that sees only the prompt can price a response it has not drawn.

8 Limitations and implications

Internal rather than external policy evaluation. All routing, learning, steering, and ex-ante outcomes are scored by the same preference model that defines the policies. Held-out likelihood shows that the choice model predicts unseen Arena battles, and the model’s predicted pairwise win rates for the ten candidate models rank them in line with their observed Arena win rates (Spearman 0.87 within the candidate set), but this is an in-sample ordering check and does not validate the counterfactual responses with new people; moreover, adding the estimated taste shocks does not improve aggregate win-rate prediction beyond logit smoothing. The paper’s contribution is consequently a structural estimate and mechanism demonstration, not an online A/B test.

Response sampling and inference cost. Generating ten candidates per request is expensive, and Section 7 shows that one draw per model is a noisy read of each model’s quality on

a prompt. We do not claim a compute-matched advantage over repeated sampling from one model, because that experiment has not been completed. The ex-ante results are the honest counterpart: they show what is reachable at one generation per request, and it is a minority of the ex-post prize unless the system knows something about the user.

Population and preference assumptions. Arena users are self-selected, anonymous, and not representative of all LLM users. We model users as draws from a stable normal taste distribution even though the data do not track a person’s repeated choices. Ordinary ties are randomized, and “both bad” identifies an outside option. Small taste spreads are weakly identified. The judged accuracy score is itself a model output, and the user-level spread of the mid-bin gain, which implies that some users value a partly right answer above a fully right one, may partly reflect judge error on answers that are hard to grade. Personalized systems can also create privacy risks by retaining preference histories and can reinforce idiosyncratic biases or reduce exposure to diverse outputs. Practical deployments should minimize stored data, permit users to inspect or reset learned profiles, and retain quality and safety constraints that personalization cannot override.

Reproducibility and deployment requirements. The replication package contains processed battle features, judged accuracy scores for every battle and every scored downstream response, model metadata, the production estimate, estimation scripts, and scripts for every reported counterfactual, table, and figure. The raw Arena releases remain available from their creators. Rebuilding the complete pipeline additionally requires access to embedding, classification, judging, and generation endpoints; closed providers can change model behavior over time. Exact response caches and model-version identifiers are therefore part of the replication artifact. Full cold-start mixed-logit estimation takes hours on a laptop; the accuracy-judging job took about 7.5 wall-clock hours on a shared compute cluster.

A deployable policy needs more than the objective studied here. It must decide whether to choose a model before generation, judge several completed responses, or combine the two in a cascade. It must price extra latency and tokens, predict correctness without access to a reference answer, handle providers entering and leaving the menu, and learn preferences without exposing sensitive prompts. The present results identify the gross value available to such a system and, in Section 6, the share of it that survives the move to one generation per request. They do not net out inference cost, and they do not establish that the ex-post ten-response policy dominates a compute-matched best-of-ten policy from one model.

Market-design interpretation. Personalization changes the value of model diversity. If assignment remains fixed, competition rewards the model with the highest average utility and can make minority tastes commercially invisible. A router gives differentiated models

demand on the prompts and users for which they have a comparative advantage. This creates option value from a broader supplier set even when one provider leads conventional benchmarks. Conversely, routing power may itself become concentrated in an intermediary whose objective, fees, or default ranking shapes traffic. Transparent utility components and user control over the learned profile are therefore economically relevant, not merely interface choices.

Within those limits, the economic conclusion is sharp. Model multiplicity has value even when average benchmark quality rises: differentiated suppliers create option value, and preference elicitation converts that option value into consumer surplus. The oracle is not a deployable number, but it is a useful target. The online-learning simulation, the verbal intervention, and the combined ex-ante router show three distinct paths toward it: learning whom to route to, adapting how a fixed model responds, and predicting before generation what each model will produce.

9 Conclusion

Users disagree about what makes an LLM response good, and those disagreements are large enough to change assignment decisions. Accuracy is the single largest term in utility, but conditional on accuracy a prompt router gains 0.31 logits over the best single model under our estimated utility function, and knowing individual tastes adds another 1.02. Simulated feedback recovers most of that incremental gain, while simple language instructions move a single model’s response toward the intended user’s preferences by as much as the oracle router does. Before generation, the prompt alone is nearly worthless for routing, but a taste vector learned from a few dozen ordinary choices, combined with predicted response characteristics, recovers 35% of the ex-post oracle without generating a single extra response. At the same time, the returns to accuracy are steeply concave, response sampling is noisy, and every downstream gain remains model-scored. Personalization therefore offers substantial economic value, but realizing it requires joint progress on preference elicitation, correctness prediction, and credible human evaluation.

References

- Bergemann, Dirk, Alessandro Bonatti, and Alex Smolin. 2025. “The Economics of Large Language Models: Token Allocation, Fine-Tuning, and Optimal Pricing.” Cowles Foundation Discussion Paper 2425, Cowles Foundation for Research in Economics, Yale University.
- Caoui, El Hadi. 2025. “Estimating Consumer Preferences for LLMs: Evidence from LMArena.” Working Paper 5970054, SSRN. 10.2139/ssrn.5970054.
- Chen, Daiwei, Yi Chen, Aniket Rege, Zhi Wang, and Ramya Korlakai Vinayak. 2025. “PAL: Sample-Efficient Personalized Reward Modeling for Pluralistic Alignment.” In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=1kFDrYCuSu>.
- Chiang, Wei-Lin, Lianmin Zheng, Ying Sheng, Anastasios N. Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael I. Jordan, Joseph E. Gonzalez, et al. 2024. “Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference.” In *Proceedings of the 41st International Conference on Machine Learning*, vol. 235 of *Proceedings of Machine Learning Research*, pp. 8359–8388.
- Demirer, Mert, Andrey Fradkin, and Nadav Tadelis. 2026. “The Emerging Market for Intelligence: How Firms Buy and Sell AI.” *Journal of Economic Perspectives* 40 (3): 23–46. 10.1257/jep.20261506.
- Garbacea, Cristina, Heran Wang, and Chenhao Tan. 2026. “Personalized Benchmarking: Evaluating LLMs by Individual Preferences.” In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 639–675. Association for Computational Linguistics. 10.18653/v1/2026.findings-acl.31.
- Kirk, Hannah Rose, Alexander Whitefield, Paul Röttger, Andrew Bean, Katerina Margatina, Juan Ciro, Rafael Mosquera, Max Bartolo, Adina Williams, He He, et al. 2024. “The PRISM Alignment Dataset: What Participatory, Representative and Individualised Human Feedback Reveals About the Subjective and Multicultural Alignment of Large Language Models.” In *Advances in Neural Information Processing Systems*, vol. 37. 10.48550/arXiv.2404.16019.
- McFadden, Daniel and Kenneth Train. 2000. “Mixed MNL Models for Discrete Response.” *Journal of Applied Econometrics* 15 (5): 447–470. 10.1002/1099-1255(200009/10)15:5<447::AID-JAE570>3.0.CO;2-1.
- Poddar, Sriyash, Yanming Wan, Hamish Ivison, Abhishek Gupta, and Natasha Jaques. 2024. “Personalizing Reinforcement Learning from Human Feedback with Variational Preference Learning.” In *Advances in Neural Information Processing Systems*, vol. 37. 10.48550/arXiv.2408.10075.
- Shenfeld, Idan, Felix Faltings, Pulkit Agrawal, and Aldo Pacchiano. 2025. “Language Model Personalization via Reward Factorization.” *arXiv preprint arXiv:2503.06358*. 10.48550/arXiv.2503.06358.

A Data construction and descriptive statistics

Table 8: Sample construction and outcomes. The all-language count is the documented pre-filter count from the two source releases; all other counts reproduce from the processed data.

Sample	Battles	Outcome	Share
Both releases, all languages	241,768	A wins	33.6%
English	128,850	B wins	34.5%
Complete features	128,384	Tie	17.2%
Both responses judged (estimation sample)	128,366	Both bad	14.7%
Training split	115,440		
Test split	12,926		

The two source datasets are the Arena 100k and 140k human-preference releases hosted by the dataset creators on Hugging Face. The English corpus is 44.8% from the 2024 release and 55.2% from the 2025 release. We remove 466 battles lacking complete response/model features or a usable outcome, and 18 further battles in which the accuracy judge did not return a score for one of the two responses. Data-processing scripts harmonize identifiers, strip conversations to the relevant prompt and response, compute deterministic features, attach model metadata, and apply a fixed train/test split. The standardization moments of the continuous features and the PLS projection of prompt embeddings are estimated on the training split only; held-out battles and all simulation prompts and responses are transformed with those training-sample quantities.

Prompts are classified into seven topics by the same classifier that labels tone and hedging. Of the 123,624 classified prompts, 37.5% are information seeking, 19.0% writing, 16.4% practical guidance, 12.7% coding, 6.7% mathematics, 3.9% casual conversation, and 3.8% multimedia. The share of “both bad” outcomes falls from 18.8% in the 2024 release to 11.4% in 2025 as the models in the arena improve. Across the 106 models in the estimation sample the benchmark intelligence index averages 43.9 (SD 13.9); among the 73 models with a matched release date, 4% were released in 2023, 47% in 2024, and 49% in 2025.

Table 9: Response-feature summary over 257,700 responses. Counts and continuous variables enter the model after the transformations described in the text; binary variables are indicators.

Feature	Mean	SD	Median	Feature	Mean	SD	Median
Length (tokens)	797.9	1200.1	504	Table	0.06	0.23	0
Ordered lists	4.15	9.74	0	Emoji	0.11	0.31	0
Unordered lists	10.43	22.44	3	Flesch ease	43.58	88.33	45.15
Headers	2.29	6.93	0	SMOG	12.79	4.12	12.70
Bold spans	13.22	27.10	5	Positive tone	0.22	0.41	0
Negative tone	0.02	0.13	0	Assertive hedge	0.47	0.50	0
Tentative hedge	0.18	0.38	0				

B Accuracy judge

The accuracy judge, `z-ai/glm-5.2` called through OpenRouter, sees the prompt and one response, truncated to the limits used in the production job, and returns a score from 0 to 10 with a short rationale, at temperature zero. On a validation sample of 50 battles (96 responses), its scores correlate at 0.80 with those from `anthropic/claude-opus-4.8`, also called through OpenRouter (Spearman 0.82, mean absolute difference 1.4 points), and the judges agree on the within-battle ordering in 95% of the 21 battles in which both are decisive; each judge agrees with the human winner in 67% and 56% of decisive battles respectively, which is a sanity check rather than a target because human winners reward style and helpfulness as well as accuracy. There are 257,682 successful production scores; 18 calls failed. The mean score is 7.34 and the standard deviation is 2.98. Among 70,208 battles with a decided or randomized outcome and a nonzero score difference, the higher-scored response wins 57.3%; among 35,632 such battles whose responses fall in different bins, the higher-bin response wins 63.1%. Excluding randomized ties, the rates are 58.7% of 57,829 and 65.4% of 30,218. On the ten-model routing panel the bin shares are 6.4%, 12.1%, and 81.6%.

C Preference estimates and identification

Table 10: Selected coefficients from the production mixed logit. Hessian standard errors are in parentheses (delta method for spreads). A taste spread at 0.020 is at the optimization lower bound and should be read as approximately zero.

Feature	Mean	Spread	Feature	Mean	Spread
Accuracy: mid	1.100 (0.026)	1.592 (0.068)	Length	0.491 (0.028)	1.066 (0.048)
Accuracy: high	1.761 (0.026)	0.027 (0.081)	Table	0.450 (0.053)	1.448 (0.178)
Emoji	0.196 (0.030)	1.432 (0.108)	Positive tone	0.110 (0.018)	0.405 (0.115)
Assertive hedge	−0.097 (0.015)	0.020 (0.049)	Tentative hedge	−0.132 (0.020)	0.699 (0.087)
Headers	0.014 (0.003)	0.092 (0.009)	Ordered lists	0.009 (0.002)	0.032 (0.005)
Bold text	0.010 (0.001)	0.020 (0.002)	Unordered lists	0.004 (0.001)	0.020 (0.002)
Flesch ease	0.003 (0.040)	0.247 (0.128)	SMOG	0.031 (0.013)	0.020 (0.036)
Negative tone	−0.075 (0.057)	0.056 (0.172)	Position B	0.016 (0.009)	—
Intelligence /10	—	0.020 (0.008)	Context window	—	0.835 (0.107)
Reasoning model	—	0.878 (0.079)			

Identification of the small spreads is weak. The spreads for unordered lists, bold text, SMOG, assertive hedging, and the intelligence index return at the 0.02 optimization floor, and the reading-ease, negative-tone, and high-accuracy spreads have standard errors of their own magnitude. The large length, table, emoji, tentative-hedging, and mid-accuracy spreads are estimated with t -statistics above 8. The Monte Carlo recovery exercise of Section 2.4 simulates 80,000 battles across 80 models from the estimated model, including the accuracy block, and re-estimates all parameters from each of 100 simulated datasets; it

Table 11: Prompt interactions for the accuracy bins (Γ) and for the style features with the largest spreads (B^R), production estimate; Hessian standard errors in parentheses. The three columns are the PLS prompt scores.

Feature	$\times$ PLS 1	$\times$ PLS 2	$\times$ PLS 3
Accuracy: mid	3.269 (0.200)	-1.389 (0.356)	0.756 (0.316)
Accuracy: high	5.262 (0.162)	-0.430 (0.286)	-0.014 (0.263)
Length	1.017 (0.161)	-3.405 (0.296)	-1.820 (0.266)
Table	-1.257 (0.366)	-1.574 (0.618)	1.911 (0.606)
Emoji	-0.117 (0.215)	-0.170 (0.383)	-0.880 (0.375)
Assertive hedge	-0.607 (0.118)	0.496 (0.198)	0.241 (0.197)
Tentative hedge	-0.229 (0.167)	-0.081 (0.269)	-0.119 (0.263)
Positive tone	-0.169 (0.138)	-0.650 (0.226)	-0.056 (0.230)
Flesch ease	-0.173 (0.286)	0.353 (0.500)	-1.827 (0.599)

is part of the replication package.

Second stage and benchmark intelligence. A second-stage OLS regression of the 73 named fixed effects on model attributes associates a ten-point increase in the benchmark index with 0.160 logits (standard error 0.030); the context-window coefficient is negative (-0.183 per million tokens, standard error 0.024), the reasoning indicator is indistinguishable from zero, and the regression explains 60% of the variance of the effects. We do not give the context-window coefficient an economic interpretation or use the second stage to rescale total utility. The benchmark index is the Artificial Analysis Intelligence Index because it covers all 106 models in the estimation sample. The choice of intelligence measure does not drive these results. Re-estimating the full model with the Epoch Capabilities Index in place of the Artificial Analysis index, on the 82,419 battles between the 73 covered models, gives a second-stage coefficient of 0.243 logits per ten index points (standard error 0.044) and the same held-out fit per battle, 0.9345. Entering the Artificial Analysis index through two bins (0.159 and 0.356 logits for the middle and top bins, standard errors 0.077 and 0.098) or adding a quadratic term (neither term significant) leaves the fit within 0.002 per battle, and the logarithm of parameter count, available for every model, gives 0.102 logits per log unit (standard error 0.019). Across the six variants the accuracy gains move by less than 0.07 logits, from 1.07 to 1.14 for the middle bin and 1.70 to 1.82 for the top bin. The full comparison is in the replication package.

Plain logit fixed effects. The association between benchmark intelligence and the model fixed effects does not depend on the random coefficients. Table 12 repeats the second stage for two plain trinomial logits estimated on the same training battles: one with model fixed effects and the position term only, and one that adds the thirteen style features and the accuracy bins with common coefficients. In each case we regress the 73 named fixed effects on the Artificial Analysis index, the context window, and the reasoning indicator. A

ten-point increase in the index is associated with 0.233 logits (standard error 0.036) when the fixed effects absorb everything, 0.123 (0.028) once style and accuracy are controlled, and 0.160 (0.030) in the mixed logit. The coefficient falls by about half when response characteristics enter, which is what one expects if part of what more capable models are rewarded for is more accurate and better formatted answers, and it is similar across the two specifications that control for them. The reasoning indicator is never distinguishable from zero. The negative context-window coefficient appears only in the mixed logit, where the context window also enters through prompt interactions and a random coefficient, which is a further reason not to interpret it.

Table 12: Second stage on the model fixed effects of three specifications estimated on the same training battles. OLS of the 73 named fixed effects on model characteristics; standard errors in parentheses.

	Logit, FE only	Logit, style and accuracy	Mixed logit
Constant	0.000 (0.142)	−0.805 (0.112)	−0.815 (0.121)
Intelligence index / 10	0.233 (0.036)	0.123 (0.028)	0.160 (0.030)
Context window (M tokens)	0.005 (0.028)	0.018 (0.022)	−0.183 (0.024)
Reasoning model	−0.130 (0.115)	−0.082 (0.090)	−0.070 (0.098)
R^2	0.53	0.33	0.60
Named models	73	73	73
Test log-likelihood	−12,776	−12,258	−12,079

A simple demand comparison. In a logit of whether model A wins a decided battle on A-minus-B differences (87,428 battles), the intelligence index alone is worth 0.2145 logits per ten points (HC1 standard error 0.005). Adding the context window, the reasoning indicator, the thirteen style differences, and the two accuracy-bin differences lowers the coefficient to 0.1660 (0.006), a 22.6% reduction, while the accuracy bins enter at 0.826 (0.022) and 1.161 (0.021) and McFadden’s R^2 rises from 0.017 to 0.070. Model fixed effects are excluded because they would absorb the time-invariant intelligence measure.

D Routing, learning, and steering details

D.1 Routing panel

The source panel begins with 5,000 topic-stratified prompts. We retain the 4,916 prompts with a response and complete features for all ten candidates. Gemini 2.5 Pro is both a candidate and the fixed baseline. The prompt router uses deterministic mean utility. The oracle adds a fixed simulated taste draw for each of 1,000 users. The same user population, seed, prompts, and candidate responses are used for both policies. Every panel response carries a judged accuracy score; the 20 responses that returned no text are scored zero, in the low bin.

The OpenRouter API identifiers used for the panel are:

GPT-4.1: openai/gpt-4.1; GPT-4o: openai/gpt-4o-2024-11-20; Claude Sonnet 4: anthropic/claude-sonnet-4; Gemini 2.5 Pro: google/gemini-2.5-pro; DeepSeek V3: deepseek/deepseek-chat-v3-0324; Llama 4 Maverick: meta-llama/llama-4-maverick; Grok 4: x-ai/grok-4; Qwen3 235B: qwen/qwen3-235b-a22b-2507; Kimi K2: moonshotai/kimi-k2; and MiniMax M1: minimax/minimax-m1.

Responses were generated in May 2026 with a 1,500-token limit, which truncated about one response in five; we regenerated those in August and September 2026 for nine of the ten models under the panel’s final generation policy (temperature 0.7, a 4,000-token limit doubled once if the response is still truncated, and a 128-token reasoning budget for Gemini 2.5 Pro so that the limit is spent on the answer). Thirteen MiniMax M1 cells remain at the original limit because the model returns no text when given a larger budget. Grok 4 had been retired by then, so its responses are frozen as first generated and 914 of them (18.6%) end at the limit. The preference model does not observe whether a response is finished, so this could bias Grok’s utility either way. It does not appear to: on the 131 prompts of similar length where Grok’s answer is complete, its gap to Gemini 2.5 Pro is 0.64 logits against 0.83 on the truncated prompts, and imputing the complete-answer gap changes the prompt-router gain by 0.005 logits and the oracle gain by 0.004.

Population shares and topic leaders. When each of 1,000 simulated users picks their utility-maximizing model on each prompt, Gemini 2.5 Pro receives 17.4% of picks (prompt-clustered standard error 0.11), Grok 4 13.4%, Qwen3 235B 11.7%, DeepSeek V3 10.7%, GPT-4.1 10.1%, MiniMax M1 8.8%, Claude Sonnet 4 8.2%, Kimi K2 7.4%, GPT-4o 6.4%, and Llama 4 Maverick 5.9%. The gap between the top two is 4.0 percentage points ($t = 28.6$). Gemini 2.5 Pro leads information seeking, practical guidance, writing, coding, and casual conversation, Qwen3 235B leads mathematics, and Grok 4 leads multimedia prompts; the leader is distinguishable from the runner-up at the 5% level in every topic except multimedia.

D.2 Online learning

For each of 200 simulated users, round t compares the model with the largest posterior mean utility to a uniformly random challenger. A/B position is randomized. The user’s A, B, or outside choice is drawn from the trinomial logit under their fixed true taste. After each response we maximize the log posterior, consisting of accumulated choice log-likelihood plus the standard-normal log prior on ν_i ; all eighteen taste components, including the two accuracy tastes, are learned. Evaluation uses 800 prompts not used for feedback.

User-level intervals in Figure 4 are computed from the cross-user sampling distribution at each checkpoint.

Table 13: Learning curve. Gap closed is relative to the prompt-router prior and the oracle on this 200-user, 800-prompt sample.

	Rounds	0	1	5	10	20	30	40
Uplift vs. Gemini 2.5 Pro		0.327	0.412	0.646	0.760	0.913	0.997	1.061
Implied win rate		54.3%	55.5%	59.2%	61.0%	63.4%	64.9%	66.0%
Oracle gap closed		0%	8.4%	31.3%	42.5%	57.6%	65.8%	72.1%

D.3 Verbal personalization

The 100 taste profiles are drawn from the production estimate: from a pool of 20,000 draws of $\nu \sim N(0, I_{18})$ we stratify the implied length coefficient into 100 quantile bins and keep the draw nearest each bin center, so the profiles span the distribution of length taste while carrying that draw’s tastes for every other feature, including the two accuracy bins. The verbalizer ranks a profile’s style deviations $\sigma^R \odot \nu_i^R$ by absolute magnitude, applies a threshold of 0.6 utility units, and emits at most four instructions. All 100 profiles receive a nonempty preamble. Each model is queried through OpenRouter under the routing panel’s generation policy (temperature 0.7, a 4,000-token limit doubled once on truncation, and a 128-token reasoning budget for Gemini 2.5 Pro) on 100 topic-stratified prompts; truncated or empty generations are retried and, if still defective, excluded, leaving between 9,619 (MiniMax M1) and 10,000 scored cells per model. Every steered and base response is scored by the accuracy judge. The utility difference is the intended user’s prompt-specific style coefficients applied to the feature differences plus the user’s own prompt-specific accuracy gain applied to the difference in accuracy bins. No model fixed effect enters because both responses come from the same model.

The prompt-clustered standard error treats the 100 prompts as independent clusters and allows arbitrary dependence across the 100 profiles within a prompt. The cross-matrix evaluates every response created for profile j under all profiles i . For Gemini 2.5 Pro its 100 diagonal profile assignments average 1.448; its 9,900 off-diagonal assignments average -0.014 . Under the average user’s tastes ($\nu = 0$) the steered responses are worth between -0.11 (DeepSeek V3) and $+0.09$ (Claude Sonnet 4) logits across the nine models.

The comparison with routing in Table 6 uses the 100 verbal prompts, which are a subset of the routing panel, and the same 100 profiles as users. Each policy is scored against Gemini 2.5 Pro’s unsteered panel response under the user’s full taste, including model fixed effects, on the 9,745 user-prompt cells in which every steered response exists. The nine-model routing rows restrict the menu to the models that can be steered; the ten-model rows add the frozen Grok 4 panel response. Fresh unsteered generations differ from the

panel response for the same prompt by -0.09 to $+0.27$ logits depending on the model, which is the sampling noise of Section 7 and not a steering effect.

E Ex-ante router details

The routers of Section 6 run on the 4,916-prompt panel with 4,000 simulated users and seed 2026. Table 14 lists every predictor. A1’s maximum-a-posteriori taste estimate uses the same trinomial likelihood and standard-normal prior as the online learner of Section 4.3, applied to k passively observed choices on learning-split prompts. A2 is a multinomial logistic regression from a 21-dimensional summary of the choice history (how often each model was offered, how often each was chosen, and how often the user rejected both) to the identity of the user’s A0-optimal model; its label accuracy on held-out users rises from 0.44 at $k = 0$ (the modal model) to 0.57 at $k = 40$. B1 and B3 are ridge regressions with per-target penalties chosen by cross-validation; B2 and B4 are 3072-512-256 networks trained with AdamW and early stopping. The routing R^2 in the table is computed after removing each model’s mean over prompts and each prompt’s mean over models, and is the only fit statistic that bears on a pick.

Table 14: Predictors estimated for the ex-ante routers. Out-of-fold R^2 on utility and on the routing-relevant prompt-by-model interaction; taste recovery is the mean correlation between the estimated and true taste vectors.

Variant	Estimator	Parameters	Fit
A1 ($k = 5/10/20/40$)	MAP posterior of ν_i	18 per user	taste recovery 0.23 / 0.31 / 0.38 / 0.47
A2 ($k = 5/10/20/40$)	multinomial logistic to model id	198	label accuracy 0.44 / 0.48 / 0.52 / 0.57
B1	ridge, embedding to utility (10 targets)	30,720	R^2 0.254, routing 0.113
B2	network, embedding to utility	1,707,274	R^2 0.293, routing 0.110
B3	ridge, embedding to 15 characteristics $\times$ 10 models	460,800	R^2 0.276 (characteristics), 0.298 (utility), routing 0.135
B4	network, embedding to characteristics	1,743,254	R^2 0.305 (characteristics), 0.304 (utility), routing 0.117

The full grid also sweeps $k \in \{0, 1, 5, 10, 20, 40\}$ for A1, A2, and C2. At $k = 0$ every learned rule collapses to the fixed baseline; at $k = 1$ the gains are 0.008 (A1), 0.001 (A2), and 0.063 (C2) logits; at $k = 5$, 0.093, 0.033, and 0.186; at $k = 20$, 0.245, 0.162, and 0.374. The number of distinct models a rule uses grows with k : A1 spreads users over 6 models from $k = 5$ on and A2 over 9 at $k = 40$, while C2 uses all ten from $k = 1$ onward because the prompt-side prediction varies the pick within a user.

F Sampling-noise audit details

The replicated grid holds 497 topic-stratified prompts drawn from the routing panel (70 to 72 per topic), the nine models that can still be queried, and five fresh OpenRouter draws per cell under the panel’s generation policy. Every replicate is judged for accuracy and scored with the production model’s deterministic-mean utility. The two-way ANOVA with interaction rejects the null of no model effect ($F = 2,084$) and of no model-by-prompt interaction tested against replicate noise ($F = 7.8$, $p < 10^{-30}$); the structural terms explain 84% of the variance and within-cell resampling the remainder. Per model, the within-cell standard deviation ranges from 0.32 logits (Claude Sonnet 4 and Llama 4 Maverick) through 0.40 (Gemini 2.5 Pro) to 0.55 (MiniMax M1). By topic, the within-cell share of variance runs from 13% (writing) to 31% (multimedia) and the interaction share from 17% (practical guidance) to 34% (mathematics). The variance shares carry prompt-cluster bootstrap 95% intervals of [14.9, 18.8] for model, [32.2, 40.4] for prompt, [24.4, 30.0] for the interaction, and [17.8, 22.1] for within-cell resampling.

The preference-learning mechanism can also create a feedback loop: models receive traffic where the router already believes they perform well, generating selective data that may reinforce initial estimates. A deployment would need randomized exploration and monitoring by prompt group to distinguish genuine specialization from a self-confirming allocation.

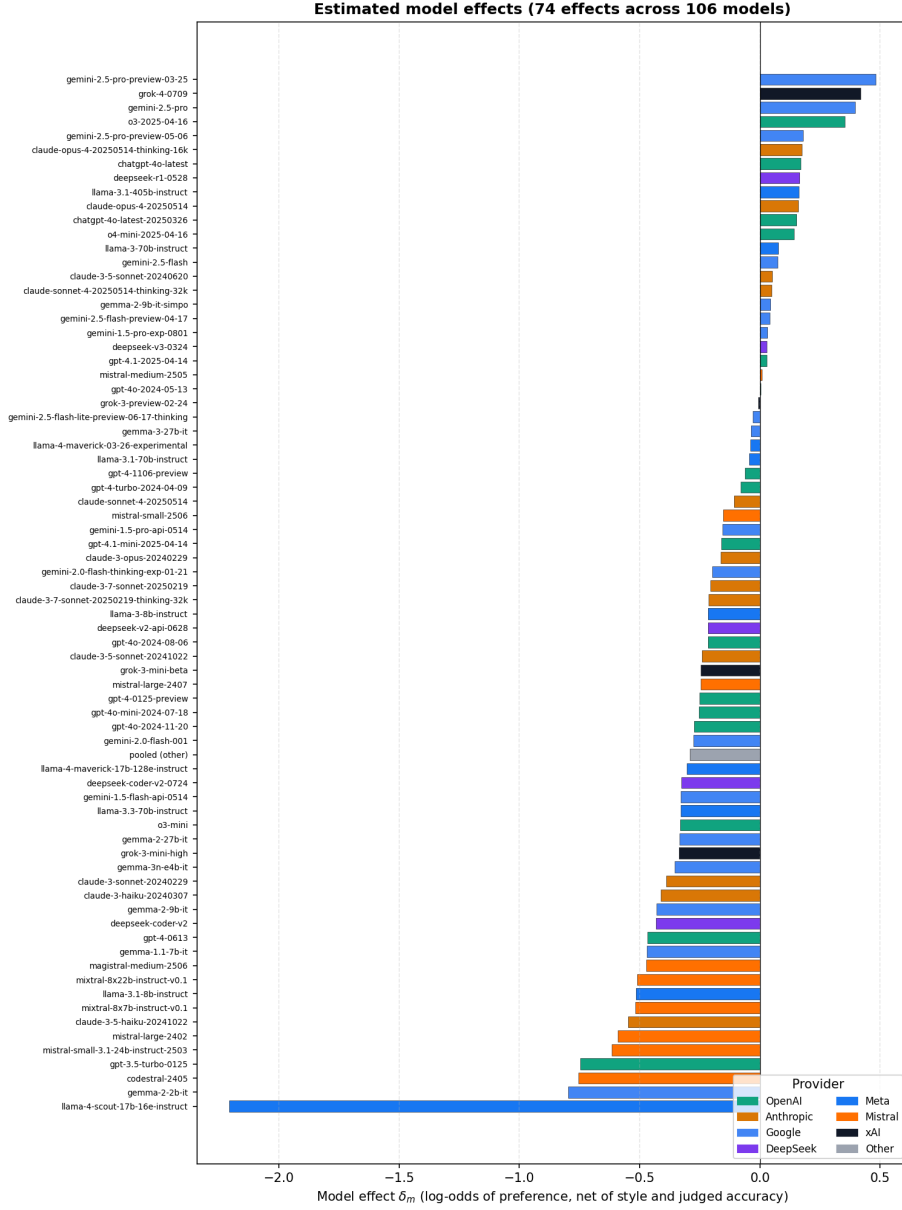


Figure 7: The 74 estimated model fixed effects (log-odds of preference, net of style and judged accuracy), colored by provider. The pooled effect covers the 33 models outside the seven largest providers.

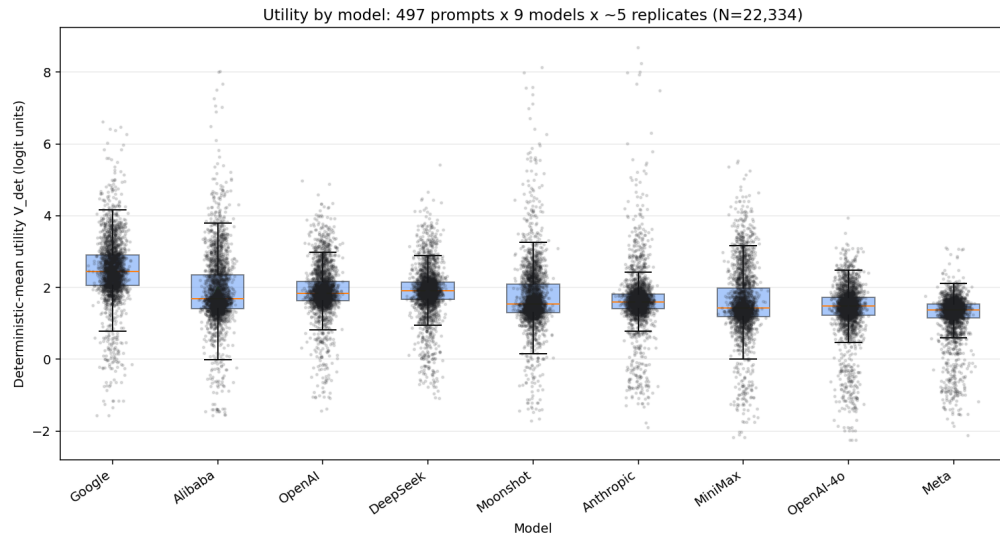


Figure 8: Deterministic-mean utility of every replicate in the sampling audit, by model. Boxes summarize the roughly 2,500 responses per model across the 497 prompts; points are individual re-draws.